David Mohaisen, Professor
University of Central Florida
Department of Computer Science
Cybersecurity and Privacy Cluster

https://cs.ucf.edu/~mohaisen/

# Training and Optimization of Neural NLP Models

① NER Classification
② Neural Scoring
③ Max-Margin Loss
④ Embedding Retraining Pitfalls
⑤ Backpropagation Theory
⑥ Backpropagation Example

# Objectives

- Rerun the binary classification for NER location
- Rerun the max-margin loss algorithm and scoring
- Pitfalls of retraining word vectors
- Backpropagation theory
- Backpropagation example

# Lecture Objectives

1. Highlight some logistics
2. Rehash of last lecture
3. Pretrained word vectors
4. Backpropogation
5. Backpropagation with an example

# Logistics

1

**Proposal Graded**

2

Assignment 1 graded/being graded

5

Office hour time; 2:00pm – 3:00pm, Tuesdays
Away: Feb 6 to Feb 12

3

**Recordings are posted on YouTube right after the lecture**

4

**Dedicate time to the project**

# Rehash

# Binary Classification for NER Location

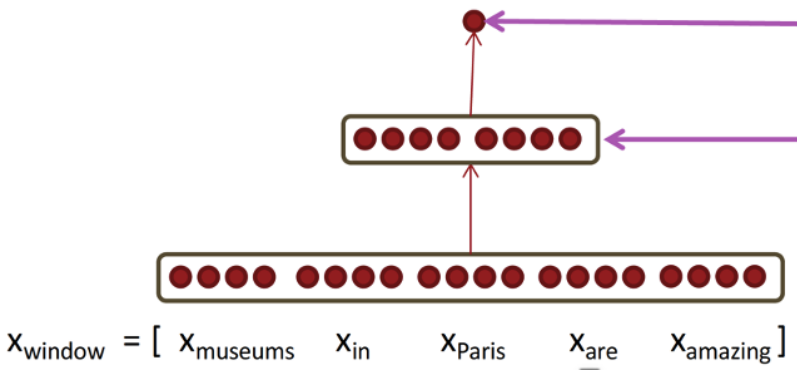
- **Sentence Variations**:
  - *Not all museums in Paris are amazing.*
  - *All museums in Paris are amazing.*
  - *Not all museums in Paris.*
- **Context Window Effect**:
  - For "museums," the window size and content affect its interpretation:
    - *Window 1*: ["Not", "all"] suggests **limitation** or **negativity**.
    - *Window 2*: ["All", "amazing"] implies **positivity** or **universality**.
    - *Window 3*: ["Not", "museums", "in"] is **incomplete**, leading to ambiguity.

# ANN Feed-Forward

- Using neural activation a simply to give an unormalized score

- We compute the wind $score(x) \quad = \quad U^T a \in \mathbb{R}$ neural network
  - s=score(“museum in Paris is amazing)

$$s = U^T f(Wx + b)$$
$$x \in \mathbb{R}^{20\times 1}, W \in \mathbb{R}^{8\times 20}, U \in \mathbb{R}^{8\times 1}$$




$$s = U^T a$$
$$a = f(z)$$
$$z = Wx + b$$

# The Max-Margin Loss

- To improve classification, we aim to increase the score of the true window ($S$) while lowering the score of the corrupted window ($S_c$).
  1. **Scores**:
     1. $S$=score("museum in Paris are amazing").
     2. $S_c$=score("not all museums in Paris").
  2. **Loss Function**:
     1. Minimize: J=max (0, 1−$S$+$Sc$)
     2. Ensures $S$ is higher than $S_c$ by a margin of 1.
  3. **Optimization**:
     1. Use Stochastic Gradient Descent (SGD) to adjust model parameters.
     2. Update only when J>0, i.e., when 1−S+Sc>0.
  - This method separates true and corrupted contexts effectively, improving classification performance.

# The Max-Margin Loss

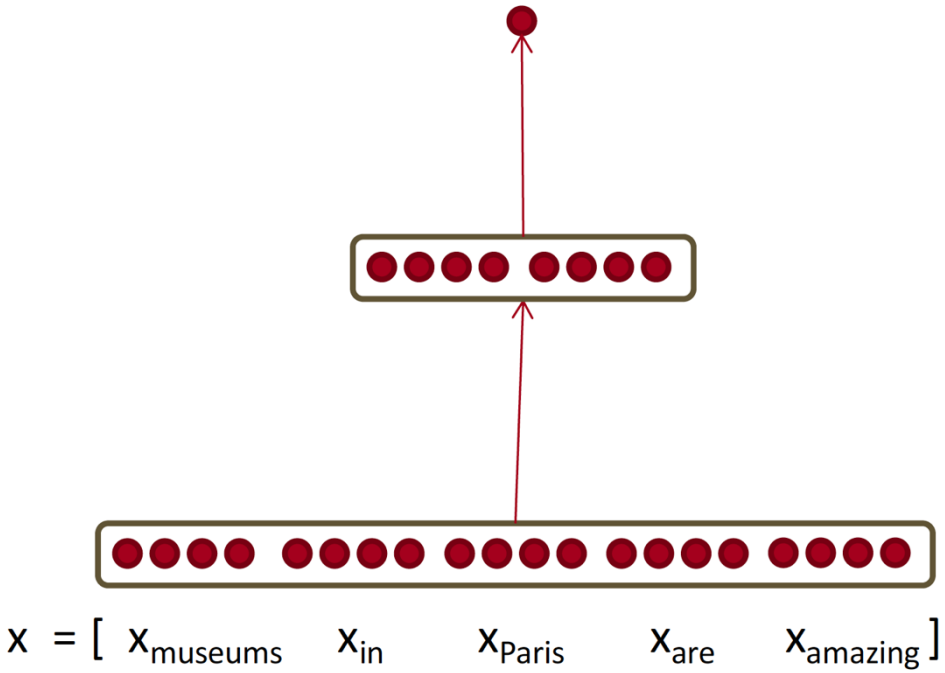
- Each context window with a Named Entity Recognition (NER) **location** at its center should have a score at least **+1 higher** than any corrupted window (i.e., one without a location at its center).
  - For each true window (containing a location entity at the center), sample multiple **corrupted windows** that lack the location entity or have incorrect contexts.

# Scoring Illustration

$$s = \boldsymbol{u}^T \boldsymbol{h}$$

$$\boldsymbol{h} = f(\boldsymbol{W}\boldsymbol{x} + \boldsymbol{b})$$

$$\boldsymbol{x} \quad \text{(input)}$$



# Pre-trained Word Vectors

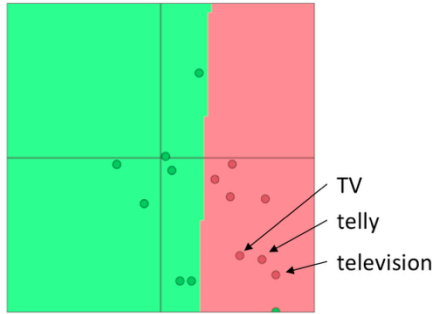
# Why Pre-trained Word Vectors Matter

- **Distributional semantics as prior knowledge**
  - Pre-trained word vectors encode semantic similarity learned from large corpora.
  - Words with similar meanings are close in embedding space.
- **Generalization through semantic geometry**
  - Models can generalize to unseen words by exploiting proximity in the embedding space.
  - This enables reasoning beyond exact word matches.
- **Key assumption:** The semantic structure of the embedding space is stable and shared across tasks.

- *This assumption holds only if the embedding space itself is not distorted during task training.*

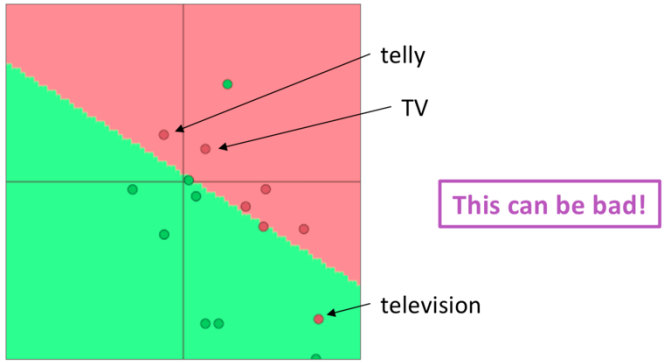
# Pitfall of Retraining Word Vectors

- **Setting:** Train a logistic regression classifier to predict the sentiment of movie reviews using individual words as input.
- **Data:** Training set contains the words *"TV"* and *"telly"*.
  - The test set contains the word *"television"*, which does not appear during training.
- **Initial state:** Pre-trained embeddings place *"TV"*, *"telly"*, and *"television"* close together due to semantic similarity.
- **What goes wrong**
  - During training, only embeddings of observed words (*"TV"*, *"telly"*) are updated.
  - These updates are driven by task-specific gradients, not semantic preservation.
  - The unseen word *"television"* remains fixed, breaking the original semantic.
- **Consequence**
  - Similar words are no longer treated similarly by the classifier.
  - Task accuracy may improve on seen words while generalization to unseen degrades.
- **Question:** What is the trade-off between task-specific fine-tuning and preserving semantic generalization?



# Pitfall of Retraining Word Vectors, cont.

- **What Happens When We Update the Word Vectors?**
  - **Words in the Training Data:** Words such as "**TV**" and "**telly**" (present in the training data) are **adjusted** during the training process. Their positions in the vector space are updated to better reflect the sentiment classification task.
  - **Words Not in the Training Data:** Words like "**television**" (absent from the training data) **retain their original positions** in the pre-trained vector space since they are not directly influenced by the updates.
- This highlights the disparity in how the model handles words it has seen versus those it has not during training.



# Pitfall of Retraining Word Vectors, cont.

- **Should We Use Pre-trained Word Vectors?**

- **Short answer: Almost always, yes. Why:**
  - Pre-trained word vectors are learned from *vast corpora*, capturing broad semantic and syntactic regularities.
  - They provide meaningful representations for words that do **not** appear in your task-specific training data.
  - Even for observed words, they typically offer **richer and more stable representations** than random initialization.

- **Exception:**
  - If you have access to **hundreds of millions (or more) tokens** of high-quality, in-domain data, learning word vectors from scratch can be a viable alternative.

# Pitfall of Retraining Word Vectors, cont.

- **Should We Fine-Tune Word Vectors?**

- **It depends on the size of the training data.**

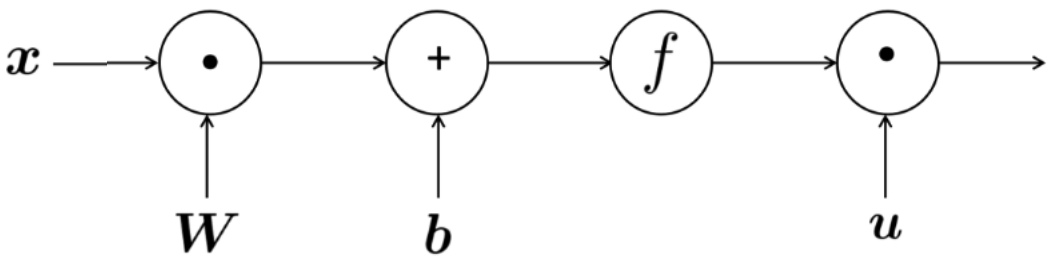
- **Small training dataset:**
  - **Do not** fine-tune the word vectors.
  - Limited data is insufficient to reliably update a high-dimensional embedding space.
  - Fine-tuning in this regime often leads to overfitting and distortion of semantic structure.
- **Large training dataset:**
  - **Fine-tune** the word vectors.
  - Sufficient data allows embeddings to adapt meaningfully to task-specific signals.
  - Task-aligned representations typically improve downstream performance.

# Backpropagation

# Backpropagation

- **Neural networks as computational graphs**
  - **Source nodes:** input variables
  - **Interior nodes:** mathematical operations
  - **Edges:** intermediate values passed between operations
- **Forward propagation**
  - Compute outputs by evaluating the graph from inputs to predictions
  - Example computation:

$$s = \boldsymbol{u}^T \boldsymbol{h}$$
$$\boldsymbol{h} = f(\boldsymbol{z})$$
$$\boldsymbol{z} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$$
$$\boldsymbol{x} \quad \text{(input)}$$



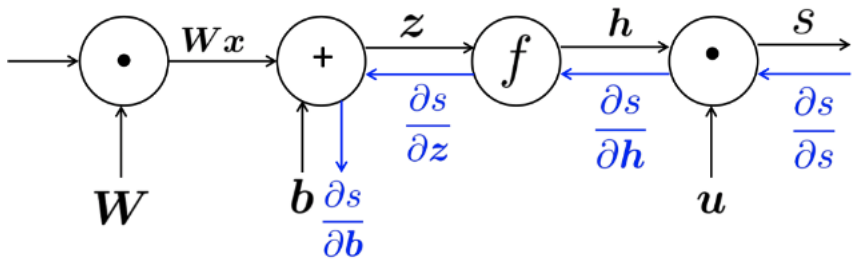

Forward Propagation

- **Why this representation matters**
  - Enables systematic application of the chain rule
  - Provides a unified framework for computing gradients with respect to all parameters

# Backpropagation

- **Backpropagation traverses the computational graph backward**
  - Gradients are propagated in the reverse direction of the forward pass
- **What is propagated**
  - Partial derivatives of the output with respect to intermediate variables
  - Ultimately yields gradients with respect to all parameters
- **Purpose**
  - Apply the chain rule efficiently across the graph
  - Compute parameter updates that **minimize the training loss**

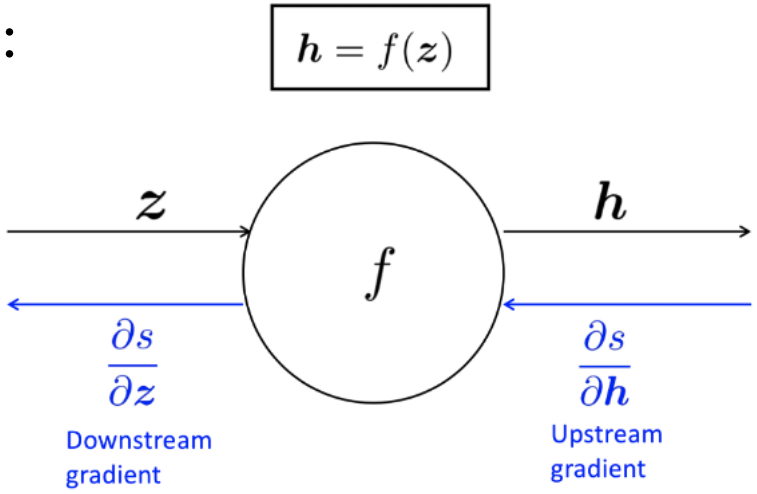
$$s = \boldsymbol{u}^T \boldsymbol{h}$$
$$\boldsymbol{h} = f(\boldsymbol{z})$$
$$\boldsymbol{z} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$$
$$\boldsymbol{x} \quad \text{(input)}$$



# Backpropagation, Single Node

- **Each node receives an upstream gradient**
  - The derivative of the loss with respect to the node's output
- **Objective:** Compute and propagate the correct downstream gradient
- **Each node has a local gradient**
  - The derivative of the node's output with respect to its input
- **Gradient propagation (chain rule)**
  - The downstream gradient is obtained by combining:
    - the upstream gradient, and
    - the node's local gradient
- **Local rule (chain rule) for** $h = f(z)$:

$$\frac{\partial s}{\partial z} = \frac{\partial s}{\partial h} \cdot \frac{\partial h}{\partial z} = \frac{\partial s}{\partial h} \cdot f'(z)$$




[downstream gradient] = [upstream gradient] x [local gradient]

# Artificial Neural Network, Recall

- We have

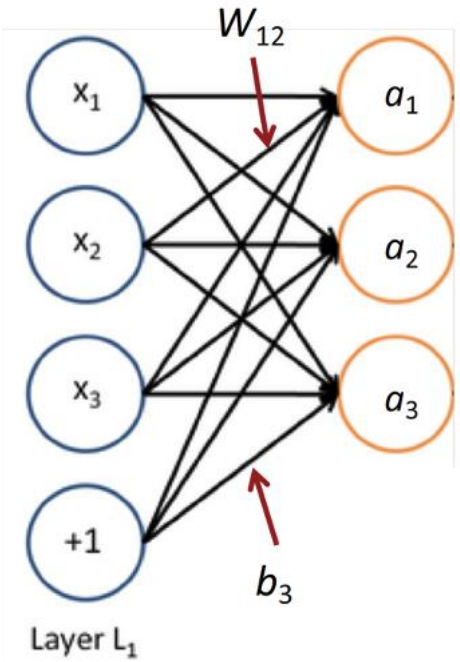
$$a_1 = f(W_{11}x_1 + W_{12}x_2 + W_{13}x_3 + b_1)$$
$$a_2 = f(W_{21}x_1 + W_{22}x_2 + W_{23}x_3 + b_2)$$
$$\text{etc.}$$

- In matrix notation

$$z = Wx + b$$
$$a = f(z)$$

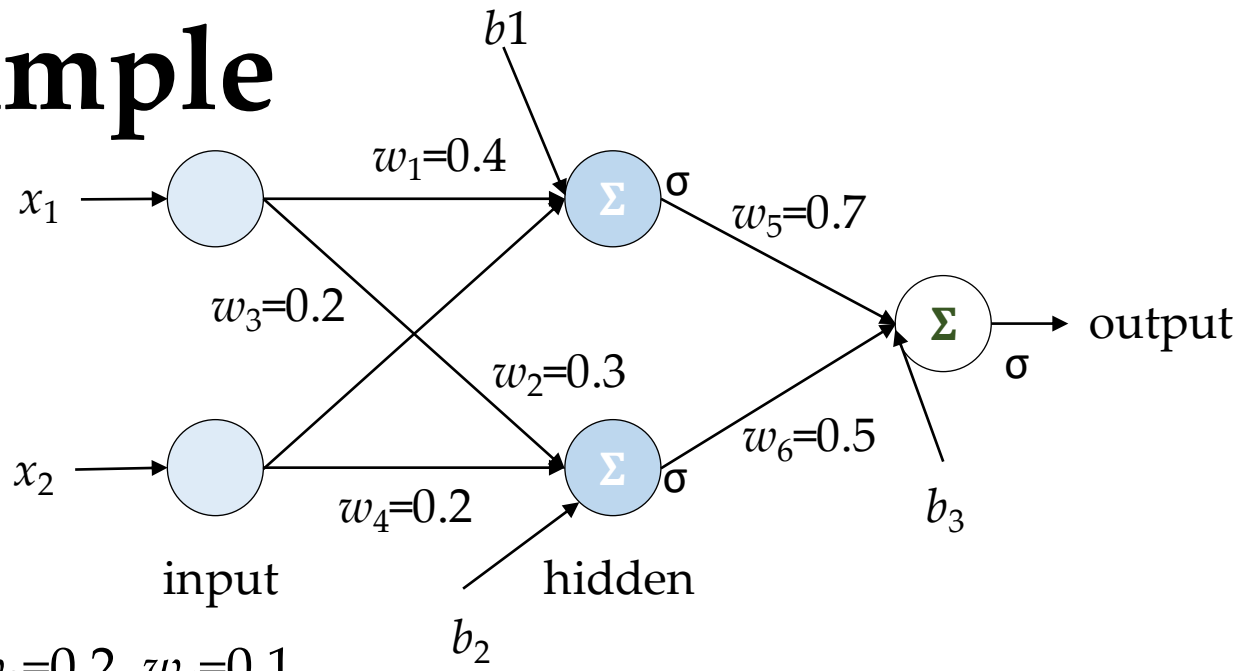
- Activation f is applied element-wise

$$f([z_1, z_2, z_3]) = [f(z_1), f(z_2), f(z_3)]$$



# Backpropagation Example

# Backpropagation: Example




- **Problem: Predict a Target Value**
- We have:
  - Input: $x_1$=1, $x_2$=2
  - Target: $y$=1
  - Weights:
    - Input to hidden layer: $w_1$=0.4, $w_2$=0.3, $w_3$=0.2, $w_4$=0.1
    - Hidden layer to output: $w_5$=0.7, $w_6$=0.5
  - Biases:
    - Hidden layer: $b_1$=0.1, $b_2$=0.2
    - Output: $b_3$=0.3
  - **Activation Function:** Sigmoid: $\sigma(z)=1/(1+e^{-z})$

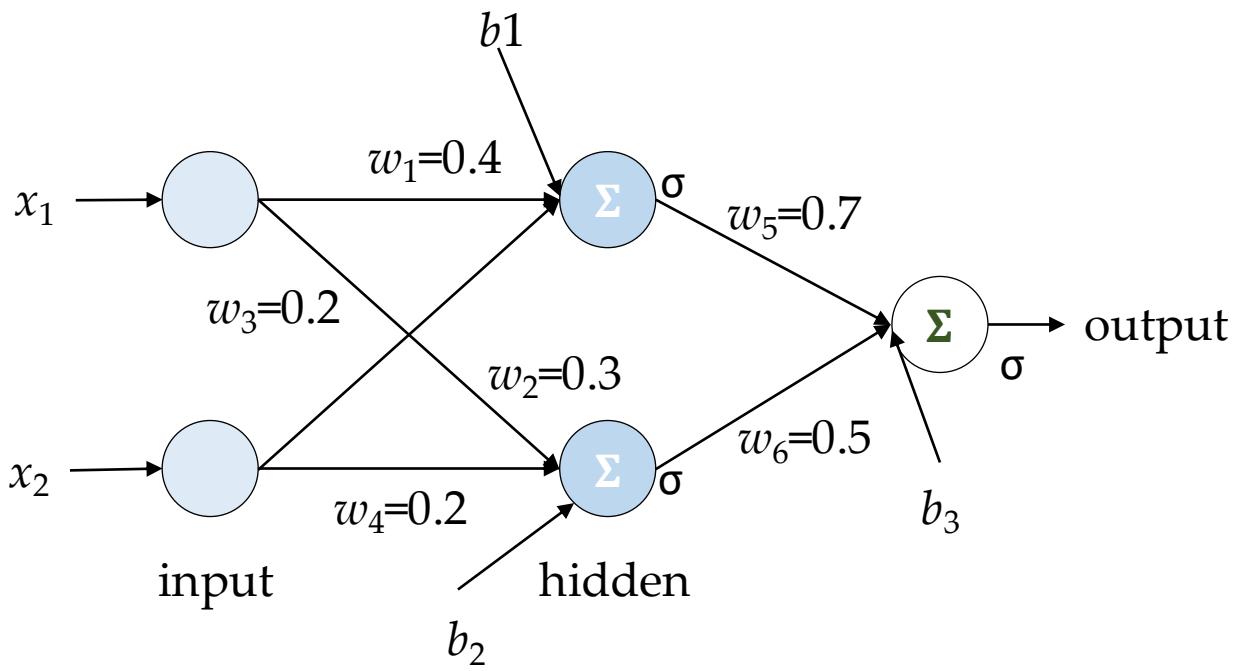
# Backpropagation: Example, cont.

- Step 1a: Hidden Layer Calculations:
  - $z_1=w_1x_1+w_2x_2+b_1=(0.4)(1)+(0.3)(2)+0.1=1.1$
    - $a_1=\sigma(z_1)=1/(1+e^{-1.1})\approx 0.75$
  - $z_2=w_3x_1+w_4x_2+b_2=(0.2)(1)+(0.1)(2)+0.2=0.6$
    - $a_2=\sigma(z_2)=1(1+e^{-0.6})\approx 0.65$

- Step 1b: Output Layer Calculation:
  - $z_3=w_5a_1+w_6a_2+b_3=(0.7)(0.75)+(0.5)(0.65)+0.3=1.085$
    - $y\hat{}=\sigma(z_3)=1/(1+e^{-1.085})\approx 0.747$

# Backpropagation: Example, cont.

- Step 2: Compute the Loss
  - Using Mean Squared Error (MSE):
    - Loss=$1/2(y-y\hat{})^2=1/2(1-0.747)^2\approx0.032$



# Backpropagation: Example, cont.

- Step 3: Backward Pass (Backpropagation)
  Goal: Update the weights using gradient descent.
- Output Layer Gradients:
  - Gradient of the loss w.r.t. $\hat{y}$:
    - $\partial \text{Loss}/\partial \hat{y} = \hat{y} - y = 0.747 - 1 = -0.253$
  - Gradient of $\hat{y}$ w.r.t. $z_3$ (sigmoid derivative):
    - $\partial \hat{y}/\partial z_3 = \hat{y}(1-\hat{y}) = 0.747(1-0.747) \approx 0.189$
  - Chain rule for $\partial \text{Loss}/\partial z_3$:
    - $\partial \text{Loss}/\partial z3 = (\partial \text{Loss}/\partial \hat{y}) \cdot \partial \hat{y}/\partial z_3 = (-0.253)(0.189) \approx -0.048$

# Backpropagation: Example, cont.

- Step 4: Hidden Layer to Output Weight Updates:
  - For $w_5$ (connected to $a_1$):
  - $\partial \text{Loss}/\partial w_5 = (\partial \text{Loss}/\partial z_3) \cdot (\partial z_3 \partial w_5) = (-0.048)(a_1) \approx -0.048(0.75) \approx -0.036$
- Update:
  - $w_5 = w_5 - \eta \cdot (\partial \text{Loss}/\partial w_5)$ (assuming $\eta = 0.01$):
  - $w_5 = 0.7 - 0.01(-0.036) = 0.7036$
- Repeat for w6w6.

# Backpropagation: Example, cont.

- Step 5: Hidden Layer Gradients:
  - Gradient of $z_3$ w.r.t. $a_1$:
    - $\partial z_3/\partial a_1 = w_5$
  - Chain rule for $\partial \text{Loss}/\partial z_1$:
    - $\partial \text{Loss}/\partial z_1 = (\partial \text{Loss}/\partial z_3)\cdot(\partial z_3/\partial a_1)\cdot(\partial a_1/\partial z_1)$.
      - Where $\partial a_1/\partial z_1 = a_1/(1-a_1)$.

# Backpropagation: Example, cont.

- Step 6: Input to hidden layer update
  - For $w_1$:
    - $\partial \text{Loss}/\partial w_1 = \partial \text{Loss}/\partial z_1 \cdot (\partial z_1/\partial w_1)$.
  - Update $w_1$:
    - $w_1 = w_1 - \eta \cdot (\partial \text{Loss}/\partial w_1)$.
- This iterative process continues until the loss converges to a minimum, effectively training the network.

Computer Understanding of
NATURAL LANGUAGE
(CAP 6640)
Q&A